

Chapter 2

Policy Engine

We create a policy engine to implement our dynamic load balancing algorithm. Each host will send state information to the policy engine. The policy engine will then make all migration decisions based on the information it receives and the policy that is currently running. By changing the policy used by the policy engine, we can implement different dynamic load balancing algorithms or even migrate for reasons other than load balancing. For example, the policy engine could be used to migrate virtual machines when hardware fails.

In Figure 2.1 we show the relationship between the host machines running Xen and the policy engine. In this Figure, there are two host machines running the Xen Virtual Machine Monitor. Each of the hosts is sending state data to the policy engine. The policy engine will tell the Xen hosts when a virtual machine should migrate to a new host depending on the state of the hosts and the policy that is being executed. In Figure 2.1, one virtual machine is currently being moved to a new host.

In the rest of this chapter, we will discuss our policy engine in greater detail.

2.1 The Specifics of Our Policy Engine

A policy engine is a software program which executes commands based on a set of rules known as a policy. In our work, the policy engine makes decisions about when to migrate virtual machines between hosts. Using a policy engine, a user can automatically migrate virtual machines to new hosts in a variety of circumstances,

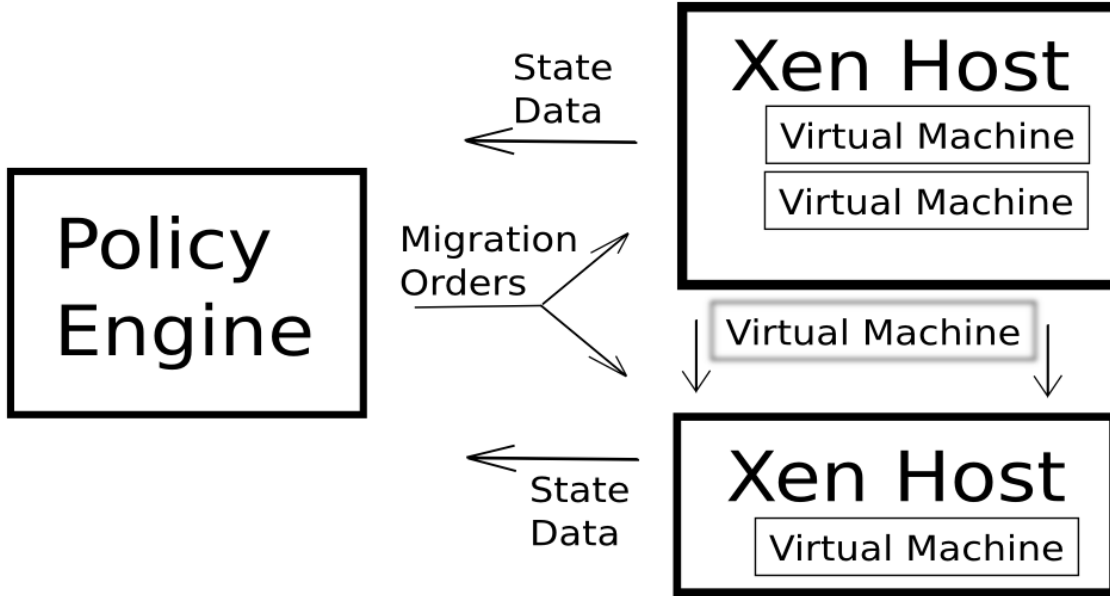


Figure 2.1: Block diagram showing relationship between policy engine and the Xen hosts.

including when hardware fails, to balance load, to place virtual machines on certain hosts in ways that might optimize performance, or to move virtual machines that should not be turned off from a host that is being shut down.

Our policy engine, which runs inside of a virtual machine using a linux kernel, queries each host for information that is relevant to the policy that is running. Relevant information could include, but is not limited to, CPU usage, memory usage, and the state of the operating system. This information is relayed through a simple web service running on each Xen host to the policy engine. Based on the information the hosts report, the policy engine will act within the parameters of its policy to decide when a virtual machine should migrate. The policy engine will also determine which host will accept the migrating virtual machines.

We configured our policy engine to dynamically load balance virtual machines based on CPU thresholds. We currently require that policies be written in Ruby, though a domain specific language may be better suited for future work. For more

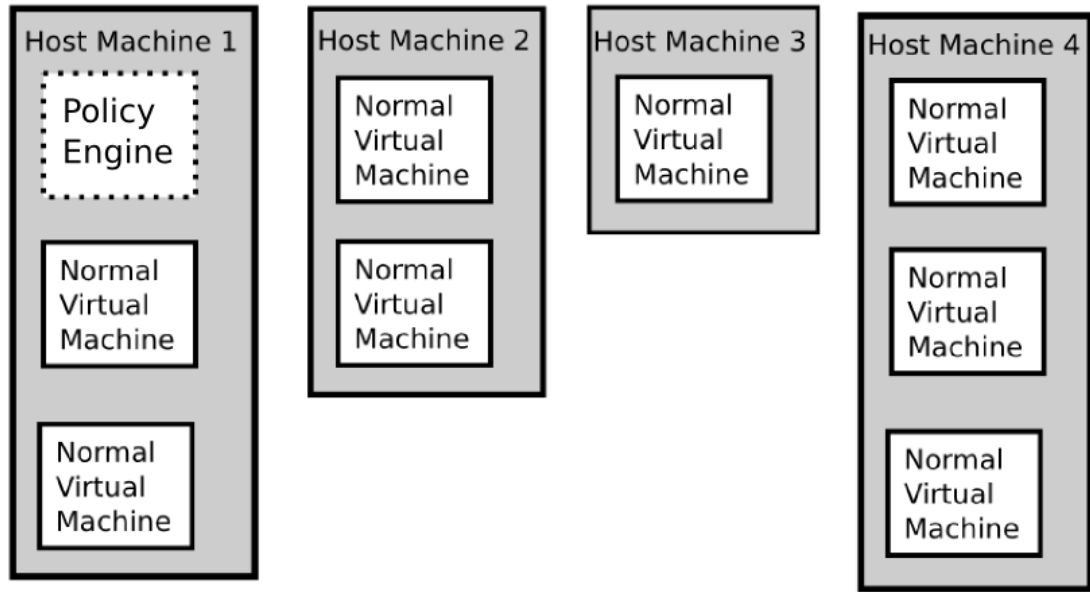


Figure 2.2: For our prototype, the policy engine runs inside of a virtual machine separate from everything else.

details about the dynamic load balancing algorithm we used for our policy, please see Section 3.

In Figure 2.2 we see an example system. Here we have four hosts, with the policy engine running on host machine 1. Please note that the policy engine is running as a normal virtual machine. From an external perspective it looks like any other virtual machine. In fact, the policy engine could even move itself to a different host like any other virtual machine.

In Figure 2.3 we see the communication links between the policy engine and the host machines. Notice that the policy engine communicates with all of the hosts. The policy engine will periodically query the hosts for information. Based on the reported information, the policy engine will instruct hosts to migrate virtual machines.

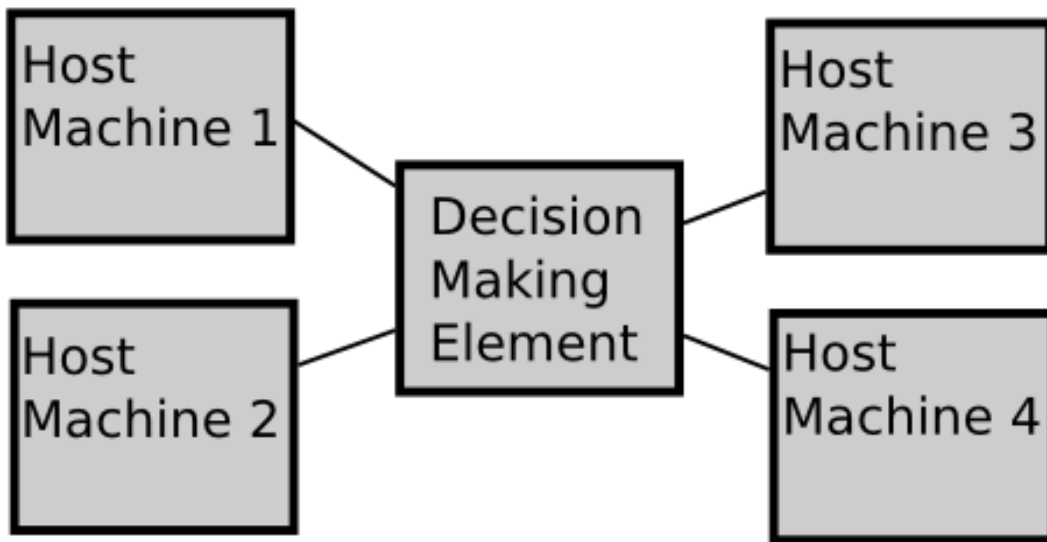


Figure 2.3: The prototype policy engine communicates with all hosts to decide when virtual machines should be migrated and to initiate migration when necessary.

Chapter 3

Load Balancing Algorithm

The algorithm we implemented inside of our policy engine is a modified version of the Central Scheduler Load Balancing (CSLB) algorithm developed by Lan and Yu [11]. We selected the CSLB algorithm because it met our requirement of being published in an academic venue while also being easy to understand and implement. Also, because the CSLB uses a central node that makes all load balancing decisions, it fits our current implementation of our policy engine better than a distributed scheduling load balancing algorithm. The differences between our algorithm and the algorithm presented by Lan and Yu will be discussed after we introduce their algorithm.

3.1 Defining the CSLB Algorithm

We will describe the CSLB algorithm using the five phases of dynamic load balancing algorithms introduced by Watts and Taylor [21]. One critical difference between the work of Watts and Taylor and our work is that Watts and Taylor were working with processes; they were not working with virtual machines. These phases are load evaluation, profitability determination, work transfer vector calculation, task selection, and task migration.

1. **Load Evaluation:** As its name implies, the CSLB is a central scheduling algorithm; all load information is sent to a central node. In the CSLB algorithm, the information to evaluate the work load can either be sent at a set interval or

when load information changes substantially. The CSLB algorithm also includes two global thresholds: one light threshold and one heavy threshold. These thresholds change based on the average CPU load of the nodes. The central scheduler computes the thresholds and broadcasts them to the other nodes. Each node determines load by using a heuristic to estimate how much time is needed to complete all of the jobs in its buffer. Each node puts itself into one of three groups based on the two thresholds and its load. Each node will then inform the central scheduler which group it is a part of. Figure 3.1 illustrates how nodes divide themselves into groups. Darker colors represent heavier loads. Based on the two thresholds created by the central scheduler, three groups are created. One group is for lightly loaded nodes; one group is for moderately loaded nodes, and the final group is for heavily loaded nodes.

2. **Profitability Determination:** Migration will be considered profitable if there is at least one host in the heavily loaded group and at least one host in the lightly loaded group.
3. **Work Transfer Vector Calculation:** Lan and Yu do not describe how to calculate the ideal amount of work to transfer in their work about the CSLB algorithm.
4. **Task Selection:** The CSLB paper also does not describe how to decide which tasks to migrate.
5. **Task Migration:** The CSLB paper does not describe how the jobs are actually transferred.

3.2 Our Modified Algorithm

Because the CSLB algorithm is not perfectly suited to our test environment, we have modified it to better meet our purposes. We describe our modified CSLB algorithm

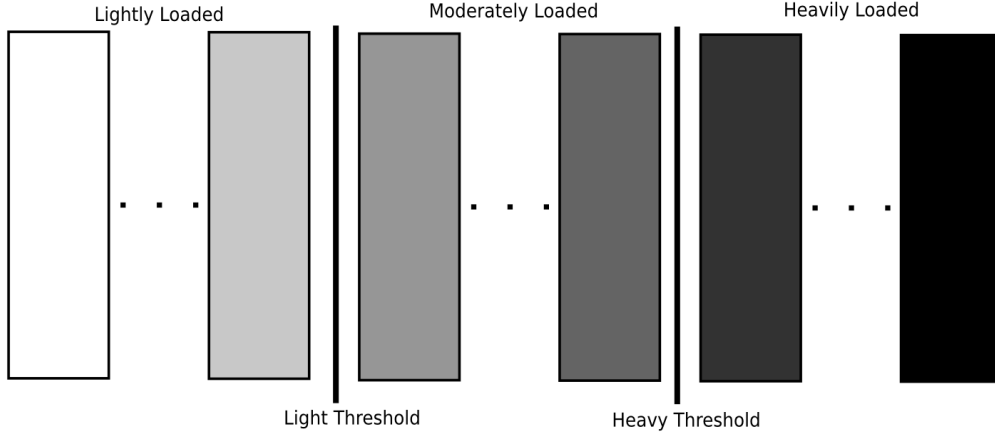


Figure 3.1: Each host machine is assigned to one of three groups based on CPU utilization and two thresholds. A darker color indicates that the host is more heavily loaded.

below using the five phases of a dynamic load balancing algorithm previously described by Watts and Taylor.

1. **Load Evaluation:** Unlike the jobs used in the original CSLB, virtual machines do not terminate. Rather, virtual machines often run indefinitely. Thus, load is determined by summing the percentage of the CPU used by each virtual machine. By changing the method that we use to compute load, we could balance load across other resources. For example, if we measured load by the amount of reads and writes to disk, we could put virtual machines that frequently use the hard disk on the same physical host as virtual machines that use the hard disk relatively infrequently. In any case, all of the workloads that we use to evaluate the dynamic load balancing of virtual machines are CPU bound. Thus, using the CPU to measure load is appropriate. Each host then puts itself into one of three groups based on two thresholds. Each host informs the central scheduler which group it is in.
2. **Profitability Determination:** Like the original CSLB algorithm, migration is considered profitable if there is at least one host in the highly utilized group and at least one host in the lightly utilized group.

3. **Work Transfer Vector Calculation:** Hosts from the lightly utilized group and heavily utilized group are paired. The ideal amount of work to transfer is equal to the difference between the middle value for the moderately utilized group and the current CPU utilization of the lightly utilized host.
4. **Task Selection:** The virtual machine from the highly utilized host with the CPU load closest to, but not exceeding, the amount specified in the work transfer vector calculation is selected for migration. If the CPU load of all virtual machines on the source host are greater than the amount specified in the work transfer vector, then the virtual machine with the smallest CPU utilization is selected.
5. **Task Migration:** The selected virtual machine is transferred to the lightly utilized host. This migration takes place using migration algorithm described Clark et. al [7].

To limit the number of variables in our test environment, our policy engine only balances load based on CPU usage. We recognize that other factors such as memory usage or I/O affect load and can contribute to load imbalances.